

อธิบายโค้ด bool

```
22 library ieee, stdcell lib;
23 use ieee.std_logic_1164.all;
24 use stdcell.lib.stdcell_pkg.all;
25 -- use ieee.numeric_std.all;
26 entity bool is
27     port(bfn      : in std_logic_vector(3 downto 0);
28           a,b      : in std_logic_vector(31 downto 0);
29           y        : out std_logic_vector(31 downto 0));
27 end entity bool;
31
32 architecture beh of bool is
33     signal yL      : std_logic_vector(31 downto 0);
34 begin -- architecture beh
35     g0: for i in 0 to 31 generate
36         yL(i) <= bfn(3) when b(i)='1' and a(i)='1' else
37                 bfn(2) when b(i)='1' and a(i)='0' else
38                 bfn(1) when b(i)='0' and a(i)='1' else
39                 bfn(0);
40     end generate g0;
41     y <= yL after tpd_mux4;
42 end architecture beh;
```

```
44 library ieee, stdcell lib;
45 use ieee.std_logic_1164.all;
46
47 architecture struct_stdcell of bool is
48
49 begin -- architecture beh
50     g0: for i in 0 to 31 generate
51         mux4_i: entity stdcell_lib.mux4(beh)
52             port map (
53                 s0 => a(i), s1 => b(i),
54                 d0 => bfn(0), d1 => bfn(1), d2 => bfn(2), d3 => bfn(3),
55                 z => y(i)
56             );
57     end generate g0;
58 end architecture struct_stdcell;
59
60
61 library ieee, stdcell lib;
62 use ieee.std_logic_1164.all;
63 use stdcell.lib.stdcell_pkg.all;
64 architecture fn_stdcell of bool is
65
66 begin -- architecture beh
67     g0: for i in 0 to 31 generate
68         y(i) <= mux4(a(i),b(i),bfn(0),bfn(1),bfn(2),bfn(3)) after tpd_mux4;
69     end generate g0;
70 end architecture fn_stdcell;
```

- 1 ส่วน Library – ทำหน้าที่เรียกใช้ไลบรารีและแพ็คเกจที่จำเป็นสำหรับการทำงานของสัญญาณดิจิทัลและ MUX4
- 2 Entity bool – ทำหน้าที่กำหนดอินพุตและเอาต์พุตของวงจร โดยรับ bfn ขนาด 4 บิต และ a, b ขนาด 32 บิต แล้วส่งผลลัพธ์ออกทาง y ขนาด 32 บิต
- 3 Architecture beh – ทำหน้าที่อธิบายการทำงานของ Boolean Logic ในรูปแบบพฤติกรรม โดยใช้ค่า a และ b เป็นตัวเลือกเพื่อเลือกค่าจาก bfn และสร้างผลลัพธ์ให้ครบทั้ง 32 บิต
- 4 for generate ใน beh – ทำหน้าที่สร้างการประมวลผล Boolean Logic สำหรับแต่ละบิต ตั้งแต่บิตที่ 0 ถึง 31
- 5 y <= yL after tpd_mux4 – ทำหน้าที่ส่งผลลัพธ์จากสัญญาณภายใน yL ไปยังเอาต์พุต y พร้อมจำลองค่าหน่วงเวลาของ MUX4
- 6 Architecture struct_stdcell – ทำหน้าที่สร้างวงจร Boolean Logic โดยใช้ MUX4 จาก Standard Cell Library แทนการเขียนเงื่อนไขโดยตรง
- 7 mux4_i – ทำหน้าที่สร้าง MUX4 สำหรับแต่ละบิต โดยใช้ a(i) และ b(i) เป็นสัญญาณเลือก และใช้ bfn(0)–bfn(3) เป็นข้อมูลอินพุตของ MUX
- 8 Architecture fn_stdcell – ทำหน้าที่สร้าง Boolean Logic โดยเรียกใช้ฟังก์ชัน mux4 โดยตรง และประมวลผลครบทั้ง 32 บิต
- 9 after tpd_mux4 ใน fn_stdcell – ทำหน้าที่กำหนดค่าหน่วงเวลาของผลลัพธ์จากการทำงานของ MUX4 เพื่อจำลองการทำงานของวงจรจริง

อธิบายโค้ด bool_tb

```

22 library ieee;
23 use ieee.std_logic_1164.all;
24 use ieee.numeric_std.all;
25 -- packages for managing text files
26 use std.textio.all;
27 use ieee.std_logic_textio.all;
28
29 entity bool_tb is
30
31 end entity bool_tb;
32
33 architecture test of bool_tb is
34     component bool is
35         port(bfn      : in std_logic_vector(3 downto 0);
36              a,b      : in std_logic_vector(31 downto 0);
37              y        : out std_logic_vector(31 downto 0));
38     end component;
39     for bool_1 : bool
40         use entity work.bool(beh);
41     for bool_2 : bool
42         use entity work.bool(struct_stdcell);
43     for bool_3 : bool
44         use entity work.bool(fn_stdcell);
45
46     -- inputs
47     -- signal clk, reset : std_logic;
48     signal bfn : std_logic_vector(3 downto 0);
49     signal a, b : std_logic_vector(31 downto 0);
50     -- outputs
51     signal y_ex : std_logic_vector(31 downto 0);
52     signal y_1, y_2, y_3 : std_logic_vector(31 downto 0);
53
54

```

```

54 constant Tclk : time := 10 ns;
55 constant Sst : time := 0.01*Tclk;
56 signal TP : integer := 0;
57 signal check : boolean;
58
59 -- function for reading expected result from the test vector file
60 function LH_to_01 (a : std_logic_vector)
61     return std_logic_vector is
62     variable y : std_logic_vector(a'range);
63 begin
64     for j in a'range loop
65         if a(j)='L' then
66             y(j) := '0';
67         elsif a(j)='H' then
68             y(j) := '1';
69         else
70             y(j) := 'U';
71         end if;
72     end loop;
73     return y;
74 end function LH_to_01;

```

```

77 begin -- architecture beh
78     bool_1: bool
79         port map (bfn => bfn, a => a, b => b, y => y_1);
80     bool_2: bool
81         port map (bfn => bfn, a => a, b => b, y => y_2);
82     bool_3: bool
83         port map (bfn => bfn, a => a, b => b, y => y_3);
84

```

```

85     process is
86         file input_file : text open read_mode is "bool_testvec.txt";
87         variable input_line : line;
88         variable bfn_v : std_logic_vector(3 downto 0);
89         variable a_v, b_v, y_v : std_logic_vector(31 downto 0);
90         variable y_string : string(33 downto 1);
91         variable y_exp : std_logic_vector(31 downto 0);
92         variable ok : boolean;
93     begin -- process
94         while (not endfile(input_file)) loop
95             TP <= TP+1;
96             readline(input_file, input_line);
97             if input_line.all'length=0 or input_line(1)='.' then
98                 next;
99             end if;
100             read(input_line,bfn_v, ok);
101             assert ok report "Read bfn failed for line" severity failure;
102             bfn <= bfn_v;
103             read(input_line,a_v, ok);
104             assert ok report "Read a_v failed for line" severity failure;
105             a <= a_v;
106             read(input_line,b_v, ok);
107             assert ok report "Read b_v failed for line" severity failure;
108             b <= b_v;
109             read(input_line,y_exp, ok);
110             assert ok report "Read y_exp failed for line" severity failure;
111             y_ex <= LH_to_01(y_exp);
112

```

```

113         -- run for Tclk*Sst, and then check the result with expected value
114         wait for Tclk*Sst;
115         assert y_1=y_ex report "Y from design#1 is wrong!" severity warning;
116         assert y_2=y_ex report "Y from design#2 is wrong!" severity warning;
117         assert y_3=y_ex report "Y from design#3 is wrong!" severity warning;
118
119         wait for Sst;
120     end loop;
121     assert false report "The function verification of BOOL module is successful!" severity note;
122     wait;
123 end process;
124 end architecture test;

```

1 เรียกใช้ Library

ใช้ Library สำหรับจัดการสัญญาณดิจิทัล ตัวเลข และการอ่านข้อมูลจากไฟล์ Text

2 สร้าง Testbench Entity

สร้าง bool_tb สำหรับทดสอบ BOOL module โดยไม่มี Input และ Output ภายนอก

3 ประกาศ Component BOOL

กำหนด Input คือ bfn ขนาด 4 bit และ a, b ขนาด 32 bit ส่วน Output คือ y ขนาด 32 bit

4 กำหนด BOOL ทั้ง 3 รูปแบบ

กำหนดให้ bool_1 ใช้ Architecture beh, bool_2 ใช้ struct_stdcell และ bool_3 ใช้ fn_stdcell เพื่อเปรียบเทียบผลลัพธ์

5 ประกาศสัญญาณ Input และ Output

กำหนด bfn, a, b เป็น Input และ y_ex, y_1, y_2, y_3 เป็น Output สำหรับใช้ตรวจสอบผล

6 ฟังก์ชัน LH_to_01

ใช้แปลงค่า L เป็น 0 และ H เป็น 1 ส่วนค่าอื่นจะถูกเปลี่ยนเป็น U

7 เชื่อมต่อ BOOL ทั้ง 3 ตัว

ส่งค่า bfn, a และ b เข้าสู่ BOOL ทั้งสามแบบพร้อมกัน และรับผลลัพธ์ออกมาเป็น y_1, y_2 และ y_3

8 อ่านข้อมูลจากไฟล์ Test Vector

เปิดไฟล์ bool_testvec.txt เพื่ออ่านค่าทดสอบ ได้แก่ bfn, a, b และ Expected Output

9 ตรวจสอบข้อมูลที่อ่าน

ใช้ assert ตรวจสอบว่าข้อมูลแต่ละค่าถูกอ่านอย่างถูกต้อง หากเกิดข้อผิดพลาดจะแจ้งเตือนและหยุดการทำงาน

10 ส่งค่า Input เข้าสู่วงจร

นำค่าที่อ่านจากไฟล์ไปกำหนดให้กับ bfn, a และ b เพื่อใช้ทดสอบ BOOL module

11 ตรวจสอบผลลัพธ์

รอให้วงจรทำงาน จากนั้นเปรียบเทียบ y_1, y_2 และ y_3 กับ y_ex หากผลไม่ตรงกันจะแสดง Warning

12 ทำการทดสอบจนครบทุกชุด

วนอ่านข้อมูลจากไฟล์และตรวจสอบผลลัพธ์จนกว่าจะถึงข้อมูลชุดสุดท้าย

13 สรุปผลการทดสอบ

เมื่อทดสอบครบทุกชุด จะแสดงข้อความแจ้งว่าการตรวจสอบฟังก์ชันของ BOOL module เสร็จสมบูรณ์